

Chapter 4

The μ ML Impact Model, v2.0

Contents

4.	The μ ML Impact Model.....	2
4.1	Elements of the Impact Model	2
4.2	Notation for the Impact Model	2
4.3	Rules of the Impact Model	3
4.4	Impact Model Examples	4
4.5	Impact Metamodel	6
4.6	Model Transformations	9
4.7	Differences from UML	10
4.8	References	10

4. The μ ML Impact Model

The μ ML *Impact Model* is constructed during requirements capture and is one of the models that must be provided from the start. When sketched by hand, an Impact Model may be constructed incrementally, which allows the engineer to identify atomic tasks and the impacts they have on objects. When formally complete, an Impact Model captures how every object is created, read, updated and deleted (or archived) by atomic tasks.

The purpose of the Impact Model is to identify atomic tasks and link these by impact transitions to the objects that they affect in various ways. Impacts are added for every atomic task that creates, reads, updates, archives or deletes an object. The Impact Model can be cross-checked against an Object Model, and also against a Task Model, for consistency. The Impact Model can be forwards-engineered as one of the sources in a transformation producing a Process Model.

4.1 Elements of the Impact Model

The Impact Model consists of the following concepts:

- **Task** – is an *atomic* unit of business activity¹, having a single impact upon some object, meaning that it creates, reads, updates or deletes that object.
- **Object** – is an atomic entity, denoting a piece of modelled data or information, that is capable of being affected by a task in the model.

The Impact Model also consists of the following connectives:

- **Impact** – a transition linking a task to an object, which describes the kind of impact the source task has, whether to *create*, *read*, *update*, *archive*, or *delete* the target object.
- **Generalisation** – linking any object to a more general object, where certain tasks may impact a more general class of objects.

The Impact Model also has the following regions:

- **Diagram** – is an implicit kind of region enclosing everything in the Impact Model. The diagram is the root container for an Impact Model.

4.2 Notation for the Impact Model

Figure 4.1 illustrates the notation used for concepts and connectives in the Impact Model. It adopts standard μ ML conventions in the concepts and connectives used.

- In particular, the task icon is the same as that used in the Task Model, and the object icon is the same rectangle as that used everywhere to indicate any object in overview.
- Likewise, the generalisation arrow is the same as that used in the Task Model and Class Model. The impact arrow is a transition arrow, with the added label denoting the kind of impact intended.

Figure 4.2 illustrates the notation used for different kinds of impact. The direction of flow is always from the task to the affected object. The commonly used kinds of impact are in the set: *{create, read, update, delete}* which represent the four main CRUD actions performed on data records. However, there are two modifications:

- An Object marked {constant} may not be updated after creation;
- An Object marked {archive} is to be archived when it is deleted.

¹ An atomic Impact corresponds to a use case in UML.

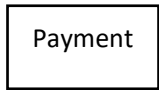
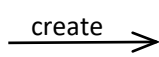
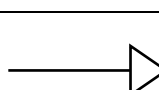
Task: denotes an <i>atomic</i> business activity with a measurable, concrete objective. Drawn as an ellipse with a solid outline, containing the name of the Task.	
Object: denotes an information entity, typically a datatype created, updated or deleted by the current system. Drawn as a rectangle containing the name of the object.	
Impact: denotes the impact that the source task has upon the target object. Drawn as a transition arrow, labelled with the kind of impact, in: {create, read, update, archive, delete}.	
Generalisation: denotes the "a kind of" relationship between a specific and a general object. The triangular arrowhead points to the general object.	

Figure 4.1: Notation for the Impact Model

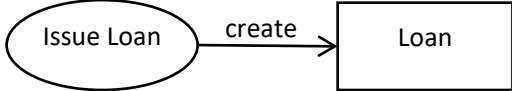
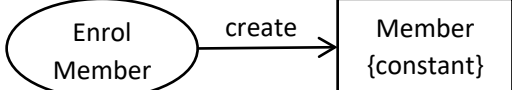
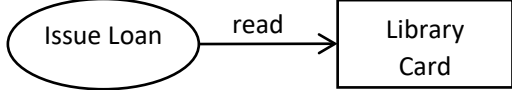
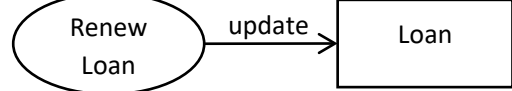
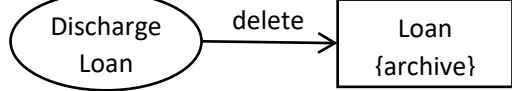
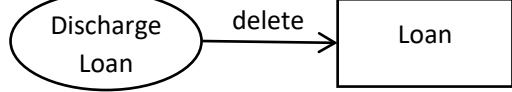
Create Impact: indicates that the task creates a new object and stores it.	
Constant Object: indicates that the task creates a constant object.	
Read Impact: indicates that the task reads the object from storage to access its data.	
Update Impact: indicates that the task reads and writes to the object and stores it.	
Archive Object: indicates that the task archives the object upon deletion.	
Delete Impact: indicates that the task deletes the object from storage.	

Figure 4.2: Meanings of different kinds of impact

4.3 Rules of the Impact Model

The Impact Model has the following syntactic and semantic rules:

- **Atomic impact** – an impact must be one which creates, deletes, updates or deletes some single piece of information (a definition rule).
- **Multiple impacts** – a task may have multiple impacts on different objects if the task requires or uses multiple objects to fulfil its objective (a definition rule).
- **Weak Completeness** – an Impact Model is *weakly* complete if every object is a target of a *create* impact and a *delete* impact (a completeness rule).

- **Strong Completeness** – an Impact Model is *strongly* complete if, in addition, every non-constant Object is the target of an *update* impact (a completeness rule).
- **Observability** – an Impact Model is observable if every object has some task which inspects it. An inspecting task has only one *read* impact on the object (a completeness rule).
- **General impact** – any impact targeting a general object is considered equivalent to an impact alternatively targeting one or other of the descendant objects (a semantic rule).
- **Object lifecycle** – the impacts on an object define its lifecycle, in which it is first created, then used (read, updated) and finally deleted or archived (a semantic rule).
- **Deletion finality** – after an object is deleted, it may not be accessed or referenced by any other task in the system; viz. it is completely removed (a semantic rule).
- **Archival persistence** – after an object is archived, it may only be accessed by tasks that inspect it, or that delete it. It cannot participate in further business activity (a semantic rule).
- **Implicit boundary** – the diagram is the implicit boundary of the Impact Model. Every contained atomic task is part of the system (a completeness rule).

A consequence of these rules is that an incomplete Impact Model could be completed by a model transformation. Completeness and observability can be achieved for any Impact Model.

4.4 Impact Model Examples

Some examples are given below of Impact Models. The first example describes a Lending Library. The second example describes a Training Agency.

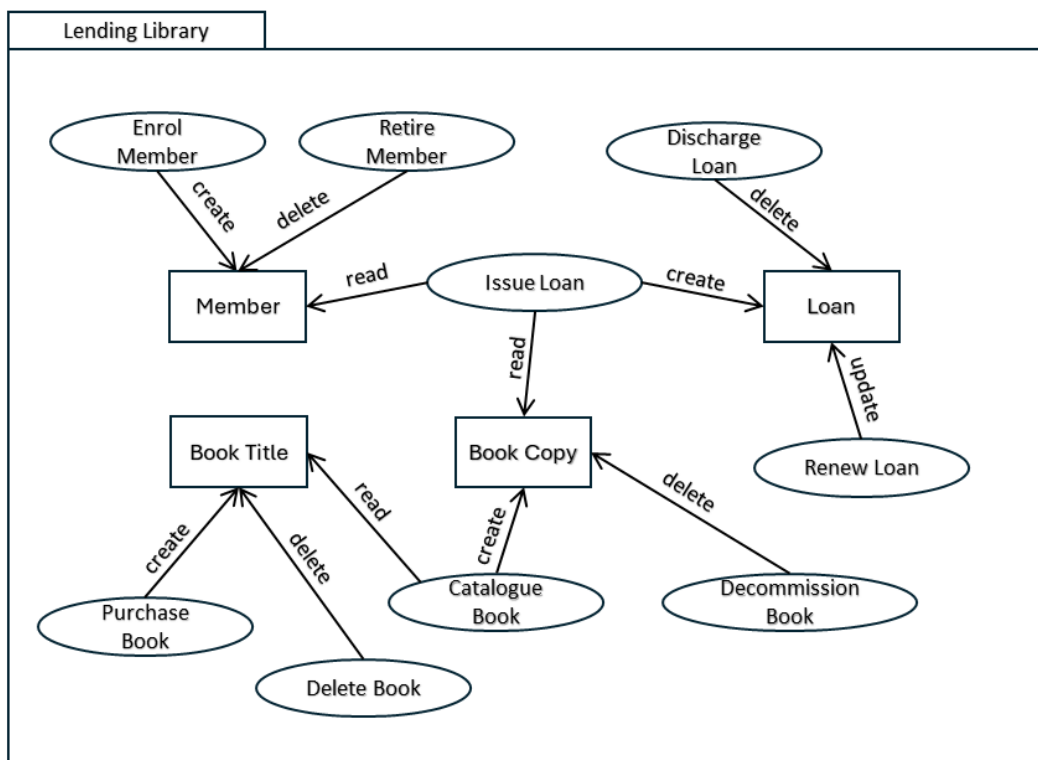


Figure 4.3: Impact Model for a Lending Library

Figure 4.3 illustrates an example Impact Model for a Lending Library, which issues loans of books to its members. The model consists of nine atomic tasks, which have different impacts upon four different objects:

- **Enrol Member** – creates a Member.
- **Retire Member** – deletes a Member.

- **Purchase Book** – creates a Book Title.
- **Delete Book** – deletes a Book Title.
- **Catalogue Book** – reads a Book Title and creates a Book Copy.
- **Decommission Book** – deletes a Book Copy.
- **Issue Loan** – reads a Member and a Book Copy and creates a Loan.
- **Renew Loan** – updates a Loan.
- **Discharge Loan** – deletes a Loan.

There are four Objects:

- **Member** – denoting a member of the library.
- **Book Title** – denoting a book in its role as a title.
- **Book Copy** – denoting an individual copy of a book.
- **Loan** – denoting the loan of a book copy to a member.

Note that the atomic tasks listed here may not be wholly consistent with those in the earlier Task Diagram: this can be repaired by a consistency-maintaining model transformation. Note that the Impact Model is weakly complete, in that every object is both created and deleted by some task, but it is not observable, since there are no tasks that simply inspect objects.

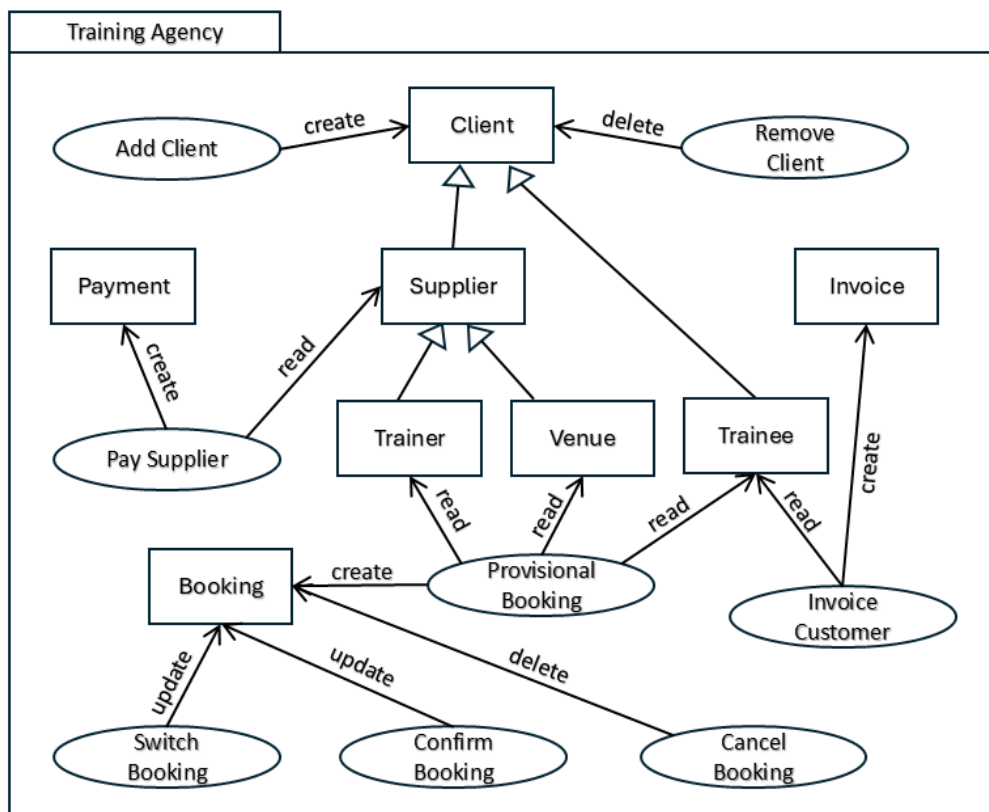


Figure 4.4: Impact Model for a Training Agency

Figure 4.4 illustrates an Impact Model for a Training Agency, which arranges training sessions for trainees with trainers, booked into different venues. The model consists of eight atomic tasks that have various impacts on eight objects:

- **Add Client** – creates a Client, which must be one of the concrete subtypes: Trainer, Venue or Trainee (by the general impact rule).

- **Remove Client** – deletes a Client, which must be one of the concrete subtypes: Trainer, Venue or Trainee (by the general impact rule).
- **Pay Supplier** – reads a Supplier (which must be one of: Venue, Trainer) and creates a Payment.
- **Invoice Customer** – reads a Trainee and creates an Invoice.
- **Provisional Booking** – reads a Trainer, a Venue and a Trainee and creates a Booking. Several provisional bookings may be made.
- **Switch Booking** – updates a Booking. Makes a different provisional booking the preferred booking (most recent preferred booking).
- **Confirm Booking** – updates a Booking. Makes a provisional booking into a definite booking.
- **Cancel Booking** – deletes a Booking.

There are eight objects, some of which are general supertypes of others:

- **Trainee** – is the main customer of the training agency, an organisation who hires training services from the agency.
- **Supplier** – is an organisation that supplies services to the training agency, who is paid for their services. A Supplier is a generalisation of either a Trainer or a Venue.
- **Trainer** – is a kind of Supplier, who offers their training services to the agency.
- **Venue** – is a kind of Supplier, who provides a conference room for the training session.
- **Client** – is any kind of organisation engaged by the training agency. A Client is a generalisation of either a Trainee, or a Trainer, or a Venue.
- **Payment** – is a payment covering note sent with a payment to a Supplier.
- **Invoice** – is an invoice for services supplied to a Trainee organisation.
- **Booking** – is a calendar booking for a Trainee, Trainer and Venue on a given date.

This example involves generalisation among some of the impacted objects. Tasks that impact on general objects must impact on one or other of the concrete subtypes. The Impact Model is clear whether a given task involves just one, or several objects. For example, the Provisional Booking task involves one each of a Trainee, Trainer and Venue. Note that this Impact Model is incomplete, since there are no tasks that delete a Payment or an Invoice; and it is also not observable.

4.5 Impact Metamodel

The elements of the μ ML Impact Model may be defined in a metamodel. In figure 4.5, we use a metamodel written in the *ReMoDeL* metamodel definition language [1]. This extends and redefines concepts defined in the ReMoDeL metamodel of the μ ML Core Model, given in Chapter 2.

```
# The Impact Model from muML (micro Modelling Language). This consists of
# Tasks which have a CRUD impact on Objects. Impact transitions run from
# the source Task to the target Object, and are labelled with the kind of
# impact, in: {create, read, update, delete}. Objects are described in
# outline only, but may be related by Generalisation. Objects may be marked
# as constant, or to be archived.

metamodel ImpactModel inherit CoreModel {

  # Vertex is an abstract kind of Concept that generalises Task and Object
  # vertices in the Impact Model. A Vertex is owned by a Diagram. A Vertex
  # offers selection operations for returning subsets of its Impacts, but
  # defers how it selects its Impacts to its descendants.

  concept Vertex inherit Concept {
    reference owner : Diagram
    operation impacts : Impact[] { null }
    operation createImpacts : Impact[] {
      self.impacts.select(imp | imp.isCreate)
    }
  }
}
```

```

    }
    operation readImpacts : Impact[] {
        self.impacts.select(imp | imp.isRead)
    }
    operation updateImpacts : Impact[] {
        self.impacts.select(imp | imp.isUpdate)
    }
    operation deleteImpacts : Impact[] {
        self.impacts.select(imp | imp.isDelete)
    }
}

# Task is a kind of Vertex representing an atomic Task, which has some
# kind of CRUD impact on an Object. A Task in the Impact Model corresponds
# to an atomic Task in the Task Model (no composite Tasks are in the Impact
# Model). However, a Task may be qualified as general, derived or simple
# according to whether its main Impact affects a supertype, subtype or
# standalone Object. A general Task denotes a choice, whose derived Tasks
# denote the branches of that choice.

concept Task inherit Vertex {
    operation impacts : Impact[] {
        owner.impacts.select(imp | imp.source = self)
    }
    operation mainImpact : Impact {
        self.createImpacts.append(self.updateImpacts)
        .append(self.deleteImpacts).append(self.readImpacts).first
    }
    operation isGeneral : Boolean {
        self.mainImpact.isGeneral
    }
    operation isDerived : Boolean {
        self.mainImpact.isDerived
    }
    operation isSimple : Boolean {
        self.mainImpact.isSimple
    }
    operation isInspect : Boolean {
        name.startsWith("View") or name.startsWith("Inspect")
    }
    operation superTasks : Task[] {
        self.mainImpact.superImpacts.collect(imp | imp.task)
    }
    operation subTasks : Task[] {
        self.mainImpact.subImpacts.collect(imp | imp.task)
    }
}

# Object is a kind of Vertex representing an information entity, which
# is affected by CRUD tasks. An Object may enter into Generalisation
# relationships with other Objects. Through this, an Object may be
# a general Object, or a derived Object. If it does not enter into any
# Generalisation, it is a simple Object. An Object may indicate that it
# is constant (upon creation) or that it is archived (upon deletion).

concept Object inherit Vertex {
    attribute constant : Boolean
    attribute archive : Boolean
    operation impacts : Impact[] {
        owner.impacts.select(imp | imp.target = self)
    }
    operation isGeneral : Boolean {
        owner.generalisations.exists(gen | gen.general = self)
    }
    operation isDerived : Boolean {
        owner.generalisations.exists(gen | gen.specific = self)
    }
    operation isSimple : Boolean {
        not owner.generalisations.exists(gen |
            gen.specific = self or gen.general = self)
    }
}

```

```

    }
    operation supertypes : Object[] {
        owner.objects.select(obj |
            owner.generalisations.exists(gen |
                gen.specific = self and gen.general = obj))
    }
    operation subtypes : Object[] {
        owner.objects.select(obj |
            owner.generalisations.exists(gen |
                gen.specific = obj and gen.general = self))
    }
}

# Impact is a kind of Connective linking a source Task to a target Object.
# The kind of the Impact indicates its CRUD effect on the target Object.
# An Impact is owned by a Diagram. The Impact may be simple, general or
# derived according to the kind of Object that it affects.

concept Impact inherit Connective {
    reference owner : Diagram
    attribute kind : String
    operation task : Task { source }
    operation object : Object { target }
    operation isCreate : Boolean { kind = "create" }
    operation isRead : Boolean { kind = "read" }
    operation isUpdate : Boolean { kind = "update" }
    operation isDelete : Boolean { kind = "delete" }
    operation isArchive : Boolean {
        self.isDelete and self.object.archive
    }
    operation isGeneral : Boolean {
        self.object.isGeneral
    }
    operation isDerived : Boolean {
        self.object.isDerived
    }
    operation isSimple : Boolean {
        self.object.isSimple
    }
    operation superImpacts : Impact[] {
        owner.impacts.select(imp | imp.kind = self.kind
            and self.object.supertypes.has(imp.object))
    }
    operation subImpacts : Impact[] {
        owner.impacts.select(imp | imp.kind = self.kind
            and self.object.subtypes.has(imp.object))
    }
    operation getName : String {
        if name /= "" then name
        else target.name.asName
    }
}

# Generalisation is a kind of Connective linking an Object to an abstract
# Object. It represents the a-kind-of relationship.

concept Generalisation inherit Connective {
    reference owner : Diagram
    operation specific : Object { source }
    operation general : Object { target }
}

# Diagram is the top-level region, enclosing the whole Impact Model.

concept Diagram inherit Region {
    component tasks : Task[]
    component objects : Object[]
    component generalisations : Generalisation[]
    component impacts : Impact[]
    operation createImpacts : Impact[] {

```



```

        impacts.select(imp | imp.isCreate)
    }
    operation readImpacts : Impact[] {
        impacts.select(imp | imp.isRead)
    }
    operation updateImpacts : Impact[] {
        impacts.select(imp | imp.isUpdate)
    }
    operation deleteImpacts : Impact[] {
        impacts.select(imp | imp.isDelete)
    }
}
}
}

```

Figure 4.5: Metamodel of the Impact Model in the ReMoDeL language.

This provides the necessary elements of the Impact Model via a number of intermediate abstractions in the metamodel hierarchy:

- **Vertex** – is a kind of Concept generalising over Task and Object, which enters into impact relationships, and is capable of filtering different kinds of Impact;
- **Task** – is a kind of Vertex, the source of Impacts;
- **Object** – is a kind of Vertex, the target of Impacts, which enters into generalisation relationships;
- **Impact** – is a kind of Connective linking source Task to a target Object;
- **Generalisation** – is a kind of Connective linking a specific and general Object;
- **Diagram** – is a kind of Region that contains all the elements of the model.

Note that Vertex provides an empty definition for its operation *impacts*. This operation is redefined in each of Task and Object to return a relevant subset of Impacts. Task and Object each have a weak back-reference *owner* to the Diagram, which allows them to search all Impacts.

4.6 Model Transformations

The μ ML Impact Model is a source or target in a number of model transformations. Some of these are to ensure completeness of the Impact Model, or enforce model consistency across different views, and some are to develop more concrete views of the software system.

- **CompleteImpactModel** – is a mapping transformation that takes an incomplete μ ML Impact Model as the source, and from this constructs a target μ ML Impact Model, which is both observable and weakly complete, with respect to *create*, *delete* and *inspect* tasks.
- **MergeObjectWithImpact** – is a merging transformation that takes an existing μ ML Impact Model and a μ ML Object Model as the sources, and from this constructs an updated target μ ML Impact Model, which is both observable and strongly complete, with respect to *create*, *delete*, *update* and *inspect* tasks.
- **ImpactToTaskModel** – is a mapping transformation that takes a μ ML Impact Model as the source, and from this constructs a default μ ML Task Model. The atomic tasks from the Impact Model are clustered under generated composite tasks, which manage the same kind of object. There is only one default user actor.
- **MergeImpactWithTask** – is a merging transformation that takes an existing incomplete μ ML Task Model and a μ ML Impact Model as the sources, and from this constructs an updated target μ ML Task Model, adding extra atomic tasks found in the Impact Model to existing composite tasks in the Task Model, where these manage the same kind of object. The actors remain the same.
- **ImpactToProcessModel** – is a mapping transformation that takes a μ ML Impact Model as the source, and from this constructs a default μ ML Process Model, which is incomplete since it

cannot reliably determine all of the input and output variables relating to the detailed view of objects. Each task is mapped to a process, which breaks down into a standard sequence of actions.

- **ImpactObjectToProcess** – is a merging transformation that takes a μ ML Impact Model and a μ ML Object Model as the sources, and from this constructs a μ ML Process Model, in which the processes are decomposed into different kinds of actions that input or output values, fetch or store objects, and create or delete objects.

These transformations are too large to display in full here.

4.7 Differences from UML

There is no direct precursor for the μ ML Impact Model, which is not found in UML. The closest work that influenced our design of the μ ML Impact Model is the Belgian software development method *MERODE* [2, 3, 4]. This highlighted the importance of CRUD operations and the notion of existence dependency when constructing object graphs. Along with the *MERODE* authors, we developed a systematic rule-based procedure for constructing Object Models that were already in 3rd Normal Form (3NF) [5].

In the *MERODE* approach, atomic business tasks are abstracted as *events*, and an Object-Event Table (OET) is constructed, which lists a set of *events* on the Y-axis and introduces *objects* along the X-axis. The engineer is encouraged to imagine which objects are involved in particular events, marking cells in the table with C, R, U, D to indicate the kind of impact. The design process ensures that create and delete events exist for known objects in the OET. Then, the design process finds sets of objects that participate in common subsets of events. This prompts the invention of a new co-ordinating object on the x-axis, whose create and delete events demarcate the lifetime of the associated object set.

Simultaneously, an Existence Dependency Graph (EDG) is constructed, which is populated initially with objects that have no dependency, but have events that affect only themselves. As further co-ordinating objects are motivated from the OET, these are added to the EDG, linking the co-ordinating object as a dependent of their co-ordinated master objects. The design process continues until all found objects are linked in the EDG, and all multiples of events are co-ordinated by some object in the OET. *MERODE* also encourages the engineer to develop simple Object State Machines (OSM), encoding the protocol for when events may be legally enacted on each object.

A much earlier approach that focused on the CRUD effects of business events upon data entities was the UK software development method SSADM [6]. This method focused on three kinds of technique: Logical Data Modelling, Data Flow Modelling and Entity Event Modelling. In the latter, the engineer was encouraged to develop Entity Life Histories (ELHs) documenting the sequence in which particular events affected each entity. The ELHs were drawn in a manner similar to a JSP diagram illustrating program structure [7] and captured notions of sequence, selection and iteration. The design of system procedures followed from a consideration of the orders in which sets of entities participated in common events.

4.8 References

- [1] AJH Simons, ReMoDeL Explained (rev 2.2): an introduction to ReMoDeL by example. *Technical Report*, 25 January, University of Sheffield (2023).
https://staffwww.dcs.shef.ac.uk/people/A.Simons/remodel/current/ReMoDeL_Explained_25Jan24.pdf
- [2] M Snoeck. *Enterprise Information Systems Engineering – The MERODE Approach*, Springer (2014).
- [3] M Snoeck, C Michiels and G Dedene. Consistency by construction: The case of MERODE. In MA Jeusfeld, O Pastor Eds., *Conceptual modeling for novel application domains, Lecture Notes in Computer Science*, **2814**, Springer (2003), 105-117.

- [4] M Snoeck and G Dedene. Existence Dependency: the key to semantic integrity between structural and behavioral aspects of object types, *IEEE Transactions on Software Engineering*, **24** (24) (1998), 233–251.
- [5] AJH Simons, M Snoeck and KSY Hung. Using design patterns to reveal the competence of object-oriented methods in system-level design, *Computer Systems Science and Engineering*, **14** (6), CRL Publishing Ltd. (1999), 343-352.
- [6] E Downs, P Claire and I Coe. *Structured Systems Analysis and Design Method: Application and Context*, 2nd Ed., Prentice Hall (1991).
- [7] MA Jackson. *Principles of Program Design*, Academic Press (1975).